

Object-oriented Programming 2 KU

Spring Boot Error Handling and Environmental Variables

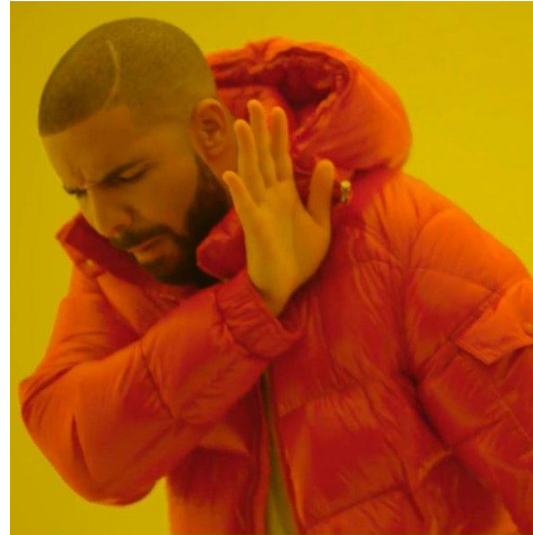
Sebastian David Albu

Institute of Computer Graphics and Knowledge Visualization (CGV)

Code on slides generated using [Carbon](#)

Why Error Handling?

“Java error messages are too long to understand anything!” -



```
java.lang.RuntimeException
  at sun.reflect.NativeConstructorAccessorImpl.newInstance0(Native Method)
  at sun.reflect.NativeConstructorAccessorImpl.newInstance(NativeConstructorAcc
  at sun.reflect.DelegatingConstructorAccessorImpl.newInstance(DelegatingConstri
  at java.lang.reflect.Constructor.newInstance(Constructor.java:513)
  at org.codehaus.groovy.reflection.CachedConstructor.invoke(CachedConstructor.:
  at org.codehaus.groovy.runtime.callsite.ConstructorSite$ConstructorSite$Outw
  at org.codehaus.groovy.runtime.callsite.CallSiteArray.defaultCallConstructor((
  at org.codehaus.groovy.runtime.callsite.AbstractCallSite.callConstructor(Abstr
  at org.codehaus.groovy.runtime.callsite.AbstractCallSite.callConstructor(Abstr
  at newifyTransform$run_closure1.doCall(newifyTransform.gdsl:121)
  at sun.reflect.NativeMethodAccessorImpl.invoke0(Native Method)
  at sun.reflect.NativeMethodAccessorImpl.invoke(NativeMethodAccessorImpl.java:
  at sun.reflect.DelegatingMethodAccessorImpl.invoke(DelegatingMethodAccessorIm
  at java.lang.reflect.Method.invoke(Method.java:1527)
  at org.codehaus.groovy.reflection.CachedMethod.invoke(CachedMethod.java:86)
  at groovy.lang.MetaMethod.doMethodInvoke(MetaMethod.java:234)
  at org.codehaus.groovy.runtime.metaclass.ClosureMetaClass.invokeMethod(Closure
  at groovy.lang.MetaClassImpl.invokeMethod(MetaClassImpl.java:893)
  at org.codehaus.groovy.runtime.callsite.PogoMetaClassSite.callCurrent(PogoMet
  at org.codehaus.groovy.runtime.callsite.AbstractCallSite.callCurrent(Abstract
  at newifyTransform$run_closure1.doCall(newifyTransform.gdsl)
  at sun.reflect.NativeMethodAccessorImpl.invoke0(Native Method)
  at sun.reflect.NativeMethodAccessorImpl.invoke(NativeMethodAccessorImpl.java:
  at sun.reflect.DelegatingMethodAccessorImpl.invoke(DelegatingMethodAccessorIm
  at java.lang.reflect.Method.invoke(Method.java:1527)
  at org.codehaus.groovy.reflection.CachedMethod.invoke(CachedMethod.java:86)
  at groovy.lang.MetaMethod.doMethodInvoke(MetaMethod.java:234)
  at org.codehaus.groovy.runtime.metaclass.ClosureMetaClass.invokeMethod(Closure
  at groovy.lang.MetaClassImpl.invokeMethod(MetaClassImpl.java:893)
  at org.codehaus.groovy.runtime.callsite.PogoMetaClassSite.call(PogoMetaClass
  at org.codehaus.groovy.runtime.callsite.AbstractCallSite.call(AbstractCallSite
  at org.jetbrains.plugins.groovy.dsl.GroovyDslExecutor$processVariants_closure
  at sun.reflect.GeneratedMethodAccessor61.invoke(Unknown Source)
  at sun.reflect.DelegatingMethodAccessorImpl.invoke(DelegatingMethodAccessorIm
  at java.lang.reflect.Method.invoke(Method.java:1527)
  at org.codehaus.groovy.reflection.CachedMethod.invoke(CachedMethod.java:86)
  at groovy.lang.MetaMethod.doMethodInvoke(MetaMethod.java:234)
  at org.codehaus.groovy.runtime.metaclass.ClosureMetaClass.invokeMethod(Closure
```



```
~$ gcc main.c
~$ ./a.out
Segmentation fault
~$
```

Benefits of Error Handling



- Ensures robust, maintainable APIs
- Provide clear error messages to users and clients
- Helps with debugging and immediate feedback in case of errors

Common Errors for Spring Boot



- Key Status Codes:
 - 400 Bad Request:
 - Invalid or missing parameters
 - 404 Not Found:
 - Requested entity not found
 - 500 Internal Server Error:
 - Unexpected server issues
 - Does this error even make sense in A0 ... ? ;)

How to check for errors?



- There are many possible ways as seen [here](#):
 - Solution 1: The Controller-Level @ExceptionHandler
 - Solution 2: The HandlerExceptionResolver
 - Solution 3: @**ControllerAdvice** 😊
 - We recommend this as it allows global exception handling across multiple controllers
 - Also:
 - Centralizes error handling logic
 - Simplifies codebase

@ControllerAdvice



```
@ControllerAdvice
public class GlobalExceptionHandler {

    @ResponseStatus(HttpStatus.NOT_FOUND)
    @ExceptionHandler(EntityNotFoundException.class)
    public ResponseEntity<ErrorResponse> handleNotFound(EntityNotFoundException ex) {
        return new ResponseEntity<>(new ErrorResponse(ex.getMessage(), 404), HttpStatus.NOT_FOUND);
    }

    // More handlers for other exceptions
}
```

@ControllerAdvice



- The @ControllerAdvice annotation allows us to consolidate our multiple, scattered @ExceptionHandler from before into a single, global error handling component.
 - It gives us full control over the body of the response as well as the status code.
 - It provides mapping of several exceptions to the same method, to be handled together.
 - It makes good use of the newer RESTful ResponseEntity response.

Custom Error Response Structure



```
public class ErrorResponse {  
    private String message;  
    private int statusCode;  
  
    public ErrorResponse(String message, int statusCode) {  
        this.message = message;  
        this.statusCode = statusCode;  
    }  
  
    // Getters and Setters  
}
```

Test with Postman or Insomnia

- Send invalid requests to test error handling (e.g., missing parameters, invalid IDs).
- View detailed JSON error responses.
- Insomnia:
 - Download Insomnia [here](#)
 - Watch the live demonstration for using Insomnia from last year [here](#)
- Postman
 - Download Postman [here](#)



POSTMAN



Insomnia

Environmental Variables

Environmental Variables



- What are environmental variables?
 - Dynamic values that configure the behavior of software.
 - Used to adapt services to different environments (e.g., dev, test, prod).
- Why they matter:
 - Centralized configuration.
 - Flexibility without code changes.

Middleware Environmental Variables



- JMAP_MIDDLEWARE_PORT:
 - Purpose: Specifies the middleware API port.
 - Handling: Convert to integer, log, and use default 8010 if invalid or missing.
- JMAP_BACKEND_TARGET:
 - Purpose: Address and port of the backend service.
 - Handling: Log value, use default localhost:8020 if not set.

Backend Environmental Variables



- JMAP_BACKEND_PORT:
 - Purpose: Specifies the backend service port.
 - Handling: Convert to integer, log, and use default 8020 if invalid or missing.
- JMAP_BACKEND_OSMFILE:
 - Purpose: Path to OpenStreetMap (OSM) data file.
 - Handling: Log value
Use default **data/styria_reduced.osm** if not set.

Get Environmental Variables



- `System.getenv().getOrDefault()`:

```
var env = System.getenv().getOrDefault("JMAP_SOMETHING", "DEFAULT:6969");
```

REMINDER:
Deadline next week!
Make sure you submit correctly!

Object-oriented Programming 2 KU

Spring Boot Error Handling and Environmental Variables

Sebastian David Albu

Institute of Computer Graphics and Knowledge Visualization (CGV)

Code on slides generated using [Carbon](#)